МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных технологий

Отчет по практической работе №1

по дисциплине «Технологии разработки программных приложений»

Тема практической работы: «Системы контроля версий»

Выполнил:
Студент группы ИКБО-50-24 Салтыков Данила Михайлович

Проверил: Старший преподаватель Золотухин С.А.

Москва 2026

Оглавление

Введение.....4

Часть 1. Начало работы с Git.....4

1.1. Установка Git.....4

1.2. Настройка Git.....4

1.3. Начало работы.....4

1.4. Создание репозитория.....6

1.5. Добавление файла в репозиторий и добавление коммита.....6

1.6. Индексация изменений.....7

1.7. Коммиты нескольких изменений.....7

1.8. Просмотр истории коммитов.....8

1.9. Получение старых версий.....9

1.10. Отмена локальных изменений (до индексации).....10

1.11. Отмена локальных изменений (после индексации и до коммита) 11

1.12. Отмена коммита.....11

Часть 2. Системы управления репозиториями.....13

2.1. Создание аккаунта на GitHub.....13

2.2. Создание SSH-ключа для авторизации.....13

2.3. Создание нового репозитория для своего проекта.....15

2.4. Связывание локального и удаленного репозитория.....15

2.5. Создание веток и переключение между ними.....16

2.6. Слияние веток.....16

2.7. Задание по варианту.....17

2.7.1. Клонирование репозитория.....17

2.7.2. Создание новой ветки и вывод всех веток.....17
2.7.3. Создание коммита в master.....18
2.7.4. Создание коммитов в новой ветке.....18
2.7.5. Выгрузка изменений в удаленный репозиторий.....19
2.7.6. Откат ветки обратно на 2 коммита.....20
2.7.7. Вывод различий между веткой main и task2.....21
2.7.8. Преобразование ветки task2 на main.....21
Часть 3. Работа с ветвлением и оформление кода.....22
3.1. Форк репозитория.....22
3.2. Клонирование репозитория.....22
3.3. Создание веток.....22
3.4. Создание коммитов в ветках.....22
3.5. Слияние ветки branch1 в ветку branch2.....23
3.6. Выгрузка всех изменений во всех ветках в удаленный репозиторий.....24
3.7. Выполнение 3-х коммитов в branch1.....25
3.8. Повторное клонирование репозитория в другую директорию....26
3.9. Выполнение 3-х коммитов в branch1.....26
3.10. Выгрузка всех изменений в удаленный репозиторий.....27
3.11. Выгрузка изменений из старого репозитория.....27
3.12. Получение всех изменений в новом репозитории.....27
4. Ответы на контрольные вопросы.....29
Вывод.....29

Введение

Цель: получить навыки по работе с командной строкой и git’ом

Часть 1. Начало работы с Git

1.1. Установка Git

На локальной машине, на которой выполняется задание уже установлен git. Проверим версию введя команду *git version*, вывод представлен на рисунке 1.1.

Рисунок 1.1 – результат выполнения команды *git version*

1.2. Настройка Git

Были введены следующие команды:

git config --global user.name "mrmin50000"

git config --global user.email "danilast2007@gmail.com"

git config --global core.autocrlf input

git config --global core.safecrlf warn

git config --global core.quotepath off

Далее для проверки работоспособности введена команда *git config –list*, ее вывод представлен на рисунке 1.2.

Рисунок 1.2 – результат выполнения команды *git config –list*

```
user.email=danilast2007@gmail.com
user.name=mrmin50000
credential.https://github.com.help
credential.https://github.com.help
credential.https://gist.github.com
credential.https://gist.github.com
core.autocrlf=input
core.safecrlf=warn
core.quotepath=off
core.bare=false
core.repositoryformatversion=0
core.filemode=true
core.logallrefupdates=true
```

1.3. Начало работы

Применим команду *cd mirea/trpp* чтобы перейти в директорию, где будут выполняться последующие задания. На рисунке 1.3 приведен результат выполнения

Рисунок 1.3 – результат выполнения команды *cd mirea/trpp*

```
~ cd mirea/trpp/
~/mirea/trpp
```

Создадим папку, в которой будет лежать репозиторий с помощью команды *mkdir practice_1*, результат выполнения представлен на рисунке 1.4.

Рисунок 1.4 – результат выполнения команды *mkdir practice_1*

```
~/mirea/trpp  mkdir practice_1
~/mirea/trpp  ls
practice_1
```

Перейдем в эту директорию и создадим файл *project.html* с помощью команды *touch project.html*, результат выполнения команды представлен на рисунке 1.5.

Рисунок 1.5 – результат выполнения команды *touch project.html*

```
~/mirea/trpp/practice_1  ls
project.html
```

Откроем редактор nvim с помощью команды *nvim project.html* и добавим текст в файл (рисунок 1.6), результат вывода содержимого с помощью команды *cat project.html* представлен на рисунке 1.7.

Рисунок 1.6 – результат ввода содержимого в html-файл через nvim

```
1  text
N  Git - Diff -  project.html[+]                    html  1:4
```

Рисунок 1.7 – результат выполнения команды *cat project.html*

```
~/mirea/trpp/practice_1  cat project.html
text
```

1.4. Создание репозитория

Создадим репозиторий с помощью команды *git init*, результат представлен на рисунке 1.8.

Рисунок 1.8 – результат выполнения команды *git init*

```
~/mirea/trpp/practice_1  git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/mrmin50000/mirea/trpp/practice_1/.git/
```

1.5. Добавление файла в репозиторий и добавление коммита

Добавим файл в репозиторий, используя команды *git add project.html* и *git commit -m “text”*, результат выполнения представлен на рисунке 1.9.

Рисунок 1.9 – результат выполнения команд *git add project.html и git commit -m “add project.html”*

```
~/mirea/trpp/practice_1  git master ?1  git add project.html
~/mirea/trpp/practice_1  git master +1  git commit -m "add project.html"
[master (root-commit) e2fdf3e] add project.html
 1 file changed, 1 insertion(+)
 create mode 100644 project.html
```

Проверим что коммитить больше нечего, используя команду *git status*, результат выполнения представлен на рисунке 1.10.

Рисунок 1.10 – результат выполнения команды *git status*

```
~/mirea/trpp/practice_1  git master  git status
On branch master
nothing to commit, working tree clean
```

Действительно, в данный момент нет никаких изменений, ожидающих записи.

Теперь внесем изменения в файл *project.html* и отобразим это используя команду *git status*, на рисунке 1.11 показано, что команда действительно отобразила изменения в файле.

Рисунок 1.11 – результат выполнения команды *git status* после изменения файла

```
 ~/mirea/trpp/practice_1  git master  nvim project.html
 ~/mi/t/practice_1  git master !1  git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   project.html

no changes added to commit (use "git add" and/or "git commit -a")
```

1.6. Индексация изменений

Для индексации изменений необходимо выполнить команды *git add project.html* и *git status*, результат выполнения представлен на рисунке 1.12.

Рисунок 1.12 – результат выполнения команды *git status* после изменения файла

```
 ~/mirea/trpp/practice_1  git master !1  git add project.html
 ~/mirea/trpp/practice_1  git master +1  git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   project.html
```

1.7. Коммиты нескольких изменений

Добавим в файл содержимое, затем добавим этот файл в staging area и потом снова внесем изменение в файл. Вывод команды *git status* представлен на рисунке 1.13.

Рисунок 1.13 – результат выполнения команды *git status* после изменения, индексации и еще одного изменения

```
~/mi/t/practice_1  git master +1 !1  git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   project.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   project.html
```

Добавим коммит и проверим состояние с помощью команды *git status*, результат выполнения представлен на рисунке 1.14.

Рисунок 1.14 – результат выполнения команды *git status* после коммита

```
~/mi/trpp/practice_1  git master !1  git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   project.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Добавим в staging area все изменения с помощью команды *git add .* и выведем статус. Результат выполнения представлен на рисунке 1.15.

Рисунок 1.15 – результат выполнения команды *git status* после добавления всех изменения в staging area

```
~/mirea/trpp/practice_1  git master +1  git add .
~/mirea/trpp/practice_1  git master +1  git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   project.html
```

Добавим коммит второго изменения, результат представлен на рисунке 1.16.

Рисунок 1.16 – результат добавления коммита

```
~/mirea/trpp/practice_1  master +1  git commit -m "modified project.html again"
[master b8e2118] modified project.html again
 1 file changed, 1 insertion(+)
~/mirea/trpp/practice_1  master  git status
On branch master
nothing to commit, working tree clean
```

1.8. Просмотр истории коммитов

Просмотрим историю коммитов с помощью команды *git log*, результат выполнения представлен на рисунке 1.17.

Рисунок 1.17 – история коммитов

```
commit b8e21188e27e486c6b3a9d324e75ccbcca656ab5 (HEAD -> master)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 09:56:21 2026 +0300

    modified project.html again

commit 8b5d6ad5d9bd4faf5d23c24c12aa598049a04225
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 09:55:13 2026 +0300

    modified project.html

commit e2fdf3ecf0ea5e97d8cfaf3fa295aa0cff66d236
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 09:49:37 2026 +0300

    add project.html
```

Просмотрим историю коммитов с помощью команды *git log*, но добавим флаг –pretty=oneline, чтобы вывести результат в одну строку, результат выполнения представлен на рисунке 1.18.

Рисунок 1.18 – история коммитов в строчку

```
b8e21188e27e486c6b3a9d324e75ccbcca656ab5 (HEAD -> master) modified project.html again
8b5d6ad5d9bd4faf5d23c24c12aa598049a04225 modified project.html
e2fdf3ecf0ea5e97d8cfaf3fa295aa0cff66d236 add project.html
```

Выведем логи с форматированием используя команду *git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short*. Здесь используются следующие форматтеры:

- %h – укороченный хэш коммита
- %d – дополнения коммита (голова веток или теги)
- %ad – дата коммита
- %s – комментарий
- %an – имя автора
- --graph – отображает дерево коммитов в виде ASCII-графика
- --date=short – сохраняет формат даты коротким

Результат выполнения представлен на рисунке 1.19.

Рисунок 1.19 – история коммитов с форматированием

```
* b8e2118 2026-02-26 | modified project.html again (HEAD -> master) [mrmin50000]
* 8b5d6ad 2026-02-26 | modified project.html [mrmin50000]
* e2fdf3e 2026-02-26 | add project.html [mrmin50000]
```

1.9. Получение старых версий

Перейдем к самому первому коммиту, его хэш – e2fdf3e. Для этого воспользуемся командой *git checkout*. Результат представлен на рисунке 1.20.

Рисунок 1.20 – результат выполнения *git checkout*.

Проверим, что содержимое файла project.html такое как было при первом коммите, результат выполнения команды *cat project.html* представлен на рисунке 1.21.

```
~/mi/t/practice_1  master  git checkout e2fdf3e
Note: switching to 'e2fdf3e'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at e2fdf3e add project.html
```

Рисунок 1.21 – результат выполнения *cat project.html*

```
~/mirea/trpp/practice_1  e2fdf3ec  cat project.html
text
```

Вернемся в нынешнее состояние. Проверим содержимое файла project.html, результат выполнения представлен на рисунке 1.22.

Рисунок 1.22 – результат выполнения *cat project.html* после возвращения к текущему коммита

```
~/mirea/trpp/practice_1  e2fdf3ec  git checkout master
Previous HEAD position was e2fdf3e add project.html
Switched to branch 'master'
~/mirea/trpp/practice_1  master  cat project.html
text
text2
text3
text4
```

1.10. Отмена локальных изменений (до индексации)

Добавим изменения в файл и проверим состояние через *git status*. Результат выполнения представлен на рисунке 1.23.

Рисунок 1.23 – результат выполнения *git status*

Выполним команду *git checkout project.html*, чтобы отменить изменения. На рисунке 1.24 представлен результат выполнения

```
~/mirea/trpp/practice_1  master  nvim project.html
~/mi/t/practice_1  master !1  git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   project.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Рисунок 1.24 – результат выполнения *git checkout project.html*

```
~/mirea/trpp/practice_1  master !1  git checkout project.html
Updated 1 path from the index
~/mirea/trpp/practice_1  master  git status
On branch master
nothing to commit, working tree clean
~/mirea/trpp/practice_1  master  cat project.html
text
text2
text3
text4
```

1.11. Отмена локальных изменений (после индексации и до коммита)

Снова внесем изменения в файл, потом проиндексируем их и проверим состояние с помощью *git status*, результат выполнения представлен на рисунке 1.25.

Рисунок 1.25 – результат выполнения *git status*

```
~/mirea/trpp/practice_1  master !1  git add .
~/mirea/trpp/practice_1  master +1  git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   project.html
```

Отменим индексацию изменений с помощью команды *git reset HEAD project.html*, результат выполнения представлен на рисунке 1.26

Рисунок 1.26 – результат выполнения *git reset HEAD project.html*

```
~/mirea/trpp/practice_1  master +1  git reset HEAD project.html
Unstaged changes after reset:
M       project.html
~/mirea/trpp/practice_1  master !1  git checkout project.html
Updated 1 path from the index
~/mirea/trpp/practice_1  master  git status
On branch master
nothing to commit, working tree clean
```

1.12. Отмена коммита

Внесем изменения в файл, проиндексируем их и напишем коммит. Результат выполнения представлен на рисунке 1.27.

Рисунок 1.26 – результат добавления изменений в файл и коммита этих изменений

```
~/mirea/trpp/practice_1  master  nvim project.html
~/mi/t/practice_1  master !1  cat project.html
text
text2
text3
text4
text5
~/mirea/trpp/practice_1  master !1  git add project.html
~/mirea/trpp/practice_1  master +1  git commit -m "modified project.html again x2"
[master d7d2dfa] modified project.html again x2
 1 file changed, 1 insertion(+)
```

Для отмены коммита выполним команду *git revert HEAD*, после ее выполнения откроется редактор, позволяющий отредактировать коммит. результат выполнения представлен на рисунках 1.27-1.29.

Рисунок 1.27 – отмена коммита

```
Revert "modified project.html again x2"

This reverts commit d7d2dfa34b68de1f5d7bdeb04f6d05155f88af00.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
# Changes to be committed:
#       modified:   project.html
#
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"/home/mrmin50000/mirea/trpp/practice_1/.git/COMMIT_EDITMSG" 11 lines, 316 characters
```

Рисунок 1.28 – сообщение о том, что коммит отменен

```
~/mirea/trpp/practice_1  git master  git revert HEAD
[master bc346b7] Revert "modified project.html again x2"
 1 file changed, 1 deletion(-)
```

Рисунок 1.29 – логи после отмены коммита

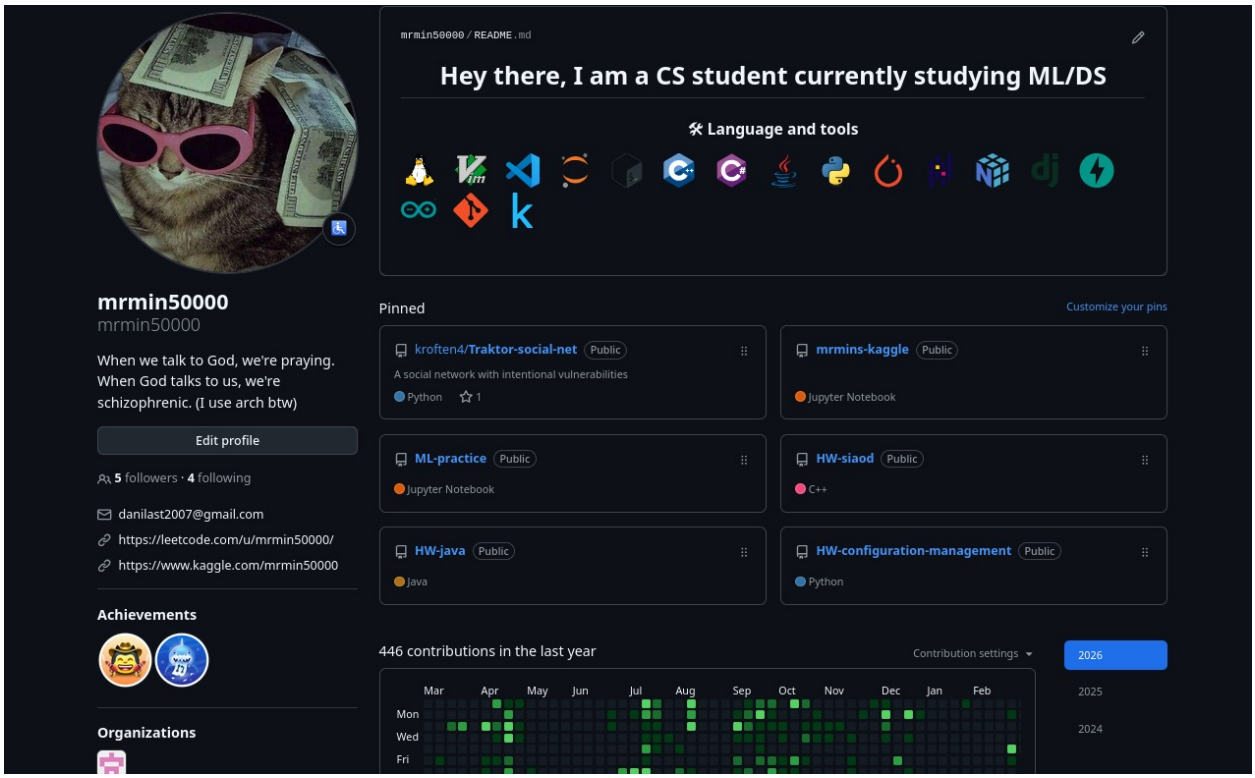
```
* bc346b7 2026-02-26 | Revert "modified project.html again x2" (HEAD -> master) [mrmin50000]
* d7d2dfa 2026-02-26 | modified project.html again x2 [mrmin50000]
* b8e2118 2026-02-26 | modified project.html again [mrmin50000]
* 8b5d6ad 2026-02-26 | modified project.html [mrmin50000]
* e2fdf3e 2026-02-26 | add project.html [mrmin50000]
```

Часть 2. Системы управления репозиториями

2.1. Создание аккаунта на GitHub

Регистрация на GitHub была выполнена, на рисунке 2.1 представлен аккаунт на GitHub.

Рисунок 2.1 – аккаунт на GitHub



2.2. Создание SSH-ключа для авторизации

Перейдем в каталог *~/.ssh* и создадим в нем ssh-ключ с помощью команды *ssh-keygen -t rsa -b 4096 -C "comment"*. Результат представлен на рисунке 2.2.

Рисунок 2.2 – создание ssh-ключа

```
~ cd ./.ssh
~/.ssh ssh-keygen -t rsa -b 4096 -C "github ssh"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/mrmin50000/.ssh/id_rsa):
Enter passphrase for "/home/mrmin50000/.ssh/id_rsa" (empty for no passphrase)
Enter same passphrase again:
Your identification has been saved in /home/mrmin50000/.ssh/id_rsa
Your public key has been saved in /home/mrmin50000/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:BPlr/h/m0nB2qb769XcUzn8g6bH8uLBl0FudhDSXKf8 github ssh
The key's randomart image is:
+---[RSA 4096]----+
|       ..     o .o|
|       ..    ..+o |
|        ..    .o. |
|        ..  .  .+.|
|         S.. ..+o+|
|         o ..*o+oE|
|        o  .B**.o.|
|         . .B*+ .=|
|          .+*O+. =|
+----[SHA256]-----+
```

Далее нужно добавить ключ в ssh-agent созданный ключ. Для этого сначала вызовем запуск ssh-agent командой *ssh-agent*, результат выполнения представлен на рисунке 2.3.

Рисунок 2.3 – запуск ssh-agent

```
~/.ssh ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-XXXXXXlcNoov/agent.29366; export SSH_AUTH_SOCK;
SSH_AGENT_PID=29367; export SSH_AGENT_PID;
echo Agent pid 29367;
```

Проверим доступность ключа командой *eval "$(ssh-agent -s)"*, кроме того, команда настроит переменные окружения для взаимодействия с ssh-agent, результат выполнения представлен на рисунке 2.4.

Рисунок 2.4 – проверка доступности ключа

```
~/.ssh eval "$(ssh-agent -s)"
Agent pid 29403
```

Добавим ssh-ключ в агент с помощью команды *ssh-add ~/.ssh/id_rsa*, результат представлен на рисунке 2.5.

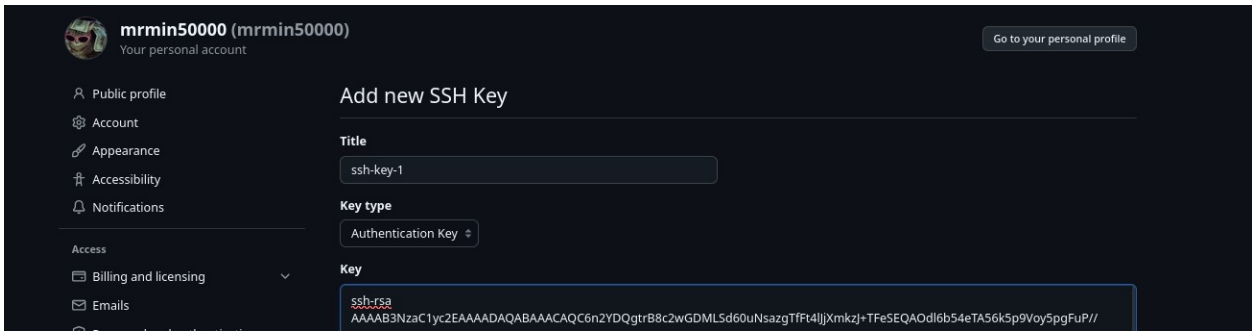
Рисунок 2.5 – добавление ssh-ключа в агента

```
~/.ssh  ssh-add ~/.ssh/id_rsa
Identity added: /home/mrmin50000/.ssh/id_rsa (github ssh)
```

Выведем содержимое публичного ключа с помощью команды *cat github.pub*, с целью безопасности в данном отчете не приведен результат выполнения данной команды, потому что показывать даже публичный ключ может быть опасно.

Далее в настройках GitHub добавим публичный ssh-ключ, на рисунке 2.6 приведен фрагмент страницы добавления ключа к GitHub аккаунту

Рисунок 2.6 – добавление ssh-ключа в GitHub



2.3. Создание нового репозитория для своего проекта

Создадим новую папку для проекта, там создадим 2 файла readme.txt и main.py, потом там же инициализируем новый локальный репозиторий, результат выполнения представлен на рисунке 2.7.

Рисунок 2.7 – создание файлов и инициализация нового локального репозитория

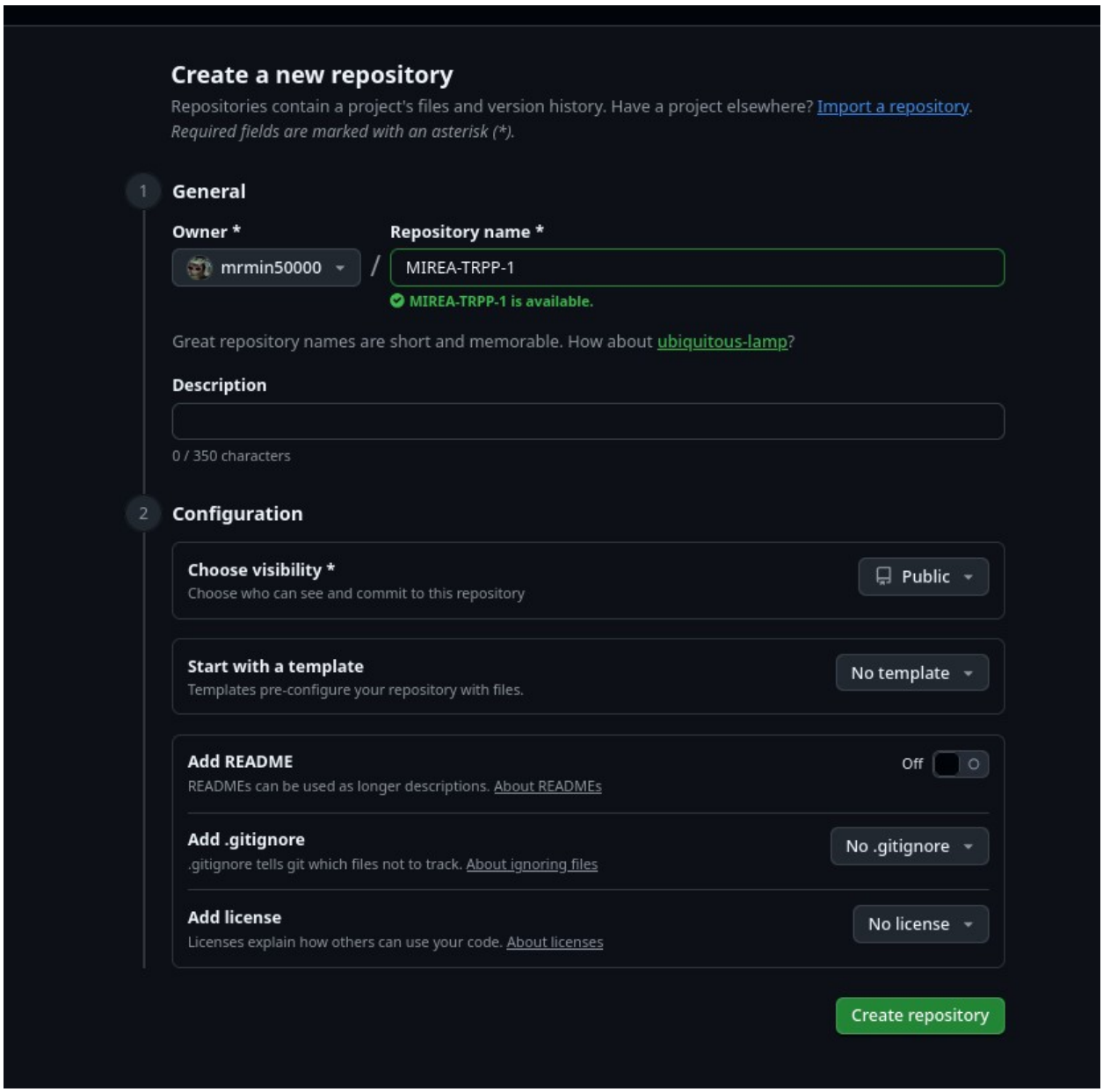
```
 ~/mirea/trpp  mkdir practice_1_2
 ~/mirea/trpp  cd practice_1_2
 ~/mirea/trpp/practice_1_2  touch readme.txt
 ~/mirea/trpp/practice_1_2  touch main.py
 ~/mirea/trpp/practice_1_2  git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/mrmin50000/mirea/trpp/practice_1_2/.git/
```

2

2.4. Связывание локального и удаленного репозитория

На GitHub на вкладке репозиториев нажмем на кнопку New, чтобы начать создавать репозиторий, на рисунке 2.8 приведена страница создания нового репозитория на GitHub.

Рисунок 2.8 – создание нового репозитория на GitHub



Теперь свяжем локальный и удаленный репозитории с помощью команды *git remote add MIREA-TRPP-1 git@github.com:TimofeyChernyshev/MIREA-TRPP-1.git*, результат выполнения представлен на рисунке 2.9.

Рисунок 2.9 – связывание локального и удаленного репозиториев

```
~/mi/trpp/practice_1_2  master ?2  git remote add project git@github.com:mrmin50000/MIREA-TRPP-1.gi
t
```

2.5. Создание веток и переключение между ними

Для создания ветки введем команду *git branch dev*, результат

выполнения представлен на рисунке 2.10.

Рисунок 2.10 – создание ветки

```
~/mi/trpp/practice_1_2  git master  git branch dev
```

Также можем создать ветку и перейти на новую ветку с помощью команды *git checkout -b main*, результат выполнения представлен на рисунке 2.11.

Рисунок 2.11 – создание и переключение на новую ветку

```
~/mirea/trpp/practice_1_2  git master  git checkout -b main
Switched to a new branch 'main'
~/mirea/trpp/practice_1_2  git main  nvim main.py
```

Добавим изменения в файл main.py и закоммитим это.

2.6. Слияние веток

Для слияния веток необходимо воспользоваться командой *git merge <название ветки>*. Перейдем в ветку master и сольем его с веткой main, в которой были произведены изменения, результат выполнения представлен на рисунке 2.12.

Рисунок 2.12 – слияние ветки master и main

```
~/mirea/trpp/practice_1_2  git main  git checkout master
Switched to branch 'master'
~/mirea/trpp/practice_1_2  git master  git merge main
Updating ffbc49a..8c092a1
Fast-forward
 main.py | 1 +
 1 file changed, 1 insertion(+)
```

2.7. Задание по варианту

В данном отчете будет выполняться вариант №7. Необходимо выполнить следующую цепочку действий в репозитории:

1) Клонируйте непустой удаленный репозиторий на локальную машину;

2) Создайте две новых ветки и выведите список всех веток

3) Произведите 3 коммита в первой ветке

4) Переместите (cherry pick) первый коммит во вторую ветку

5) Выгрузите изменения в удаленный репозиторий

6) Выведите в консоли различия между веткой master и новыми ветками

7) Слейте обе ветки с master при помощи merge

2.7.1. Клонирование репозитория

Чтобы клонировать репозиторий воспользуемся командой *git clone git@github.com:mrmin50000/HW-java.git*, результат выполнения представлен на рисунке 2.13.

Рисунок 2.13 – клонирование удаленного репозитория

```
 ~/mirea/trpp/practice_1_2  git  master  ..
 ~/mirea/trpp  git clone git@github.com:mrmin50000/HW-java.git
Cloning into 'HW-java'...
The authenticity of host 'github.com (140.82.121.3)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 224, done.
remote: Counting objects: 100% (224/224), done.
remote: Compressing objects: 100% (156/156), done.
remote: Total 224 (delta 74), reused 184 (delta 42), pack-reused 0 (from 0)
Receiving objects: 100% (224/224), 48.97 KiB | 343.00 KiB/s, done.
Resolving deltas: 100% (74/74), done.
```

2.7.2. Создание двух новых веткок и вывод всех веток

Создадим новые ветки task1 и task2 с помощью команды *git branch task1 и git branch task2*, результат создания приведен на рисунке 2.14.

Рисунок 2.14 – создание новой ветки

```
 ~/mirea/trpp/HW-java  main  git branch task1
 ~/mirea/trpp/HW-java  main  git branch task2
```

Теперь выведем список всех веток в репозитории с помощью команды *git branch*, результат выполнения приведен на рисунке 2.15.

Рисунок 2.15 – вывод всех существующих веток

```
* main
  task1
  task2
```

2.7.3. Создание 3 коммитов в main

Результат выполнения показан на рисунке 2.16.

Рисунок 2.16

```
 ~/mirea/trpp/HW-java  git  task1  nvim README.md
 ~/mirea/trpp/HW-java  git  task1 !1  git add README.md
 ~/mirea/trpp/HW-java  git  task1 +1  git commit -m "1"
[task1 535f58c] 1
 1 file changed, 1 insertion(+), 1 deletion(-)
 ~/mirea/trpp/HW-java  git  task1  nvim README.md
 ~/mirea/trpp/HW-java  git  task1 !1  git add README.md
 ~/mirea/trpp/HW-java  git  task1 +1  git commit -m "2"
[task1 5f2ec60] 2
 1 file changed, 1 insertion(+), 1 deletion(-)
 ~/mirea/trpp/HW-java  git  task1  nvim README.md
 ~/mirea/trpp/HW-java  git  task1 !1  git add README.md
 ~/mirea/trpp/HW-java  git  task1 +1  git commit -m "3"
[task1 abc21c2] 3
 1 file changed, 1 insertion(+), 1 deletion(-)
```

2.7.4. Cherry pick

результат выполнения приведен на рисунке 2.18.

Рисунок 2.18

```
commit abc21c220f18b5564ea3c418de6824bdd6fb28b8 (HEAD -> task1)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:05:24 2026 +0300

    3

commit 5f2ec60f1a9198442fff9b5957403e419d02a08a
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:05:16 2026 +0300

    2

commit 535f58c6e40d20622b8b3a1cfab4fb744287afde
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:05:09 2026 +0300

    1
```

```
~/mirea/trpp/HW-java  task1  git log
~/mirea/trpp/HW-java  task1  git switch task2
Switched to branch 'task2'
~/mirea/trpp/HW-java  task2  git cherry-pick 535f58c6e40d20622b8b3a1cfab4fb744287afde
[task2 063e3b1] 1
 Date: Thu Feb 26 11:05:09 2026 +0300
 1 file changed, 1 insertion(+), 1 deletion(-)
~/mirea/trpp/HW-java  task2
```

2.7.5. Выгрузка изменений в удаленный репозиторий

Для выгрузки изменений в удаленный репозиторий воспользуемся командой *git push --all*, где branch – имя ветки, которую необходимо выгрузить на GitHub. Выгрузим изменения из веток main и task2, результат выполнения приведен на рисунке 2.19.

Рисунок 2.19

```
~/mirea/trpp/HW-java  task2  git push --all
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (10/10), 762 bytes | 762.00 KiB/s, done.
Total 10 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
To github.com:mrmin50000/HW-java.git
 * [new branch]      task1 -> task1
 * [new branch]      task2 -> task2
~/mirea/trpp/HW-java  task2
```

2.7.6. Различие веток

результат представлен на рисунке 2.20.

Рисунок 2.20

```
* commit 063e3b1b78a122fc2379121394e392e50b617e63 (HEAD -> task2, origin/task2)
| Author: mrmin50000 <danilast2007@gmail.com>
| Date:   Thu Feb 26 11:05:09 2026 +0300
|
|     1
|
| * commit abc21c220f18b5564ea3c418de6824bdd6fb28b8 (origin/task1, task1)
| | Author: mrmin50000 <danilast2007@gmail.com>
| | Date:   Thu Feb 26 11:05:24 2026 +0300
| |
| |     3
| |
| * commit 5f2ec60f1a9198442fff9b5957403e419d02a08a
| | Author: mrmin50000 <danilast2007@gmail.com>
| | Date:   Thu Feb 26 11:05:16 2026 +0300
| |
| |     2
| |
| * commit 535f58c6e40d20622b8b3a1cfab4fb744287afde
|/  Author: mrmin50000 <danilast2007@gmail.com>
|   Date:   Thu Feb 26 11:05:09 2026 +0300
|
|       1
|
* commit 3d7bd275b722e650cbc39795e07653b73053276c (origin/main, origin/HEAD, main)
| Author: mrmin50000 <danilast2007@gmail.com>
| Date:   Thu Feb 26 10:59:16 2026 +0300
|
|     4
|
```

2.7.7. merge

результат выполнения представлен на рисунке 2.22.

Рисунок 2.22

```
*   commit a6a50886e42730ede52785c5f8a4d88abeab0f9d (HEAD -> main, task2)
|\  Merge: 063e3b1 abc21c2
| | Author: mrmin50000 <danilast2007@gmail.com>
| | Date:   Thu Feb 26 11:12:08 2026 +0300
| |
| |     merge
| |
| * commit abc21c220f18b5564ea3c418de6824bdd6fb28b8 (origin/task1, task1)
| | Author: mrmin50000 <danilast2007@gmail.com>
| | Date:   Thu Feb 26 11:05:24 2026 +0300
| |
| |     3
| |
| * commit 5f2ec60f1a9198442fff9b5957403e419d02a08a
| | Author: mrmin50000 <danilast2007@gmail.com>
| | Date:   Thu Feb 26 11:05:16 2026 +0300
| |
| |     2
| |
| * commit 535f58c6e40d20622b8b3a1cfab4fb744287afde
| | Author: mrmin50000 <danilast2007@gmail.com>
| | Date:   Thu Feb 26 11:05:09 2026 +0300
| |
| |     1
| |
* | commit 063e3b1b78a122fc2379121394e392e50b617e63 (origin/task2)
|/  Author: mrmin50000 <danilast2007@gmail.com>
|   Date:   Thu Feb 26 11:05:09 2026 +0300
```

Часть 3. Работа с ветвлением и оформление кода

Эта часть выполняется по варианту 8, необходимо работать с репозиторием по ссылке https://github.com/Sairyss/domain-driven-hexagon.

3.1. Форк репозитория

Чтобы создать форк репозитория нажмем на кнопку Fork в репозитории, который собираемся форкать.

3.2. Клонирование репозитория

Клонируем репозиторий на компьютер с помощью команды *git clone*, результат приведен на рисунке 3.1.

Рисунок 3.1 – клонирование репозитория

```
~/mirea/trpp  git clone https://github.com/mrmin50000/domain-driven-hexagon.git
Cloning into 'domain-driven-hexagon'...
remote: Enumerating objects: 3360, done.
remote: Counting objects: 100% (594/594), done.
remote: Compressing objects: 100% (109/109), done.
remote: Total 3360 (delta 521), reused 485 (delta 485), pack-reused 2766 (from 2)
Receiving objects: 100% (3360/3360), 7.61 MiB | 76.00 KiB/s, done.
Resolving deltas: 100% (1952/1952), done.
```

3.3. Создание веток

Создадим две ветки, которые назовем branch1 и branch2 с помощью команды *git branch <имя ветки>* и выведем список всех веток, он представлен на рисунке 3.2.

Рисунок 3.2 – список веток в репозитории

```
~/mirea/trpp/domain-driven-hexagon  master  git branch branch1
~/mirea/trpp/domain-driven-hexagon  master  git branch branch2
```

3.4. Создание коммитов в ветках

Теперь необходимо создать по 3 коммита в каждой ветке, которые меняют одну и ту же часть файла. На рисунке 3.3 представлен список коммитов для ветки branch1, а на рисунке 3.4 список коммитов для ветки branch2. Они все меняют один и тот же кусок кода в файле README.md

Рисунок 3.3 – список коммитов в ветке branch1

```
commit a944a6ee64884a79271746bb99a95d8c9ac90ce5 (HEAD -> branch1)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:25 2026 +0300

    3

commit 07893ccdf1c2e12f9c08bb7dcad8de79004fc79f
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:20 2026 +0300

    2

commit b8177f4415b4b047af1177e4549090d3ed62ed2c
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:07 2026 +0300

    1
```

Рисунок 3.4 – список коммитов в ветке branch2

```
commit 2088d593ae1e328d212bfac6b8d90009c58e5fdd (HEAD -> branch2)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:24:09 2026 +0300

    3

commit 93aebcb5333b888c769f51f7fc03354ee404e137
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:24:04 2026 +0300

    2

commit 736513aee56cd2c89356b84184d8470f4e750da3
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:58 2026 +0300

    1
```

3.5. Слияние ветки branch1 в ветку branch2

Рисунок 3.5

```
~/mi/t/domain-driven-hexagon  branch2  git merge branch1
Merge made by the 'ort' strategy.
~/mi/trpp/domain-driven-hexagon  branch2
```

3.6. Выгрузка всех изменений во всех ветках в удаленный репозиторий

Git push --all

Рисунок 3.8 – выгрузка изменений из веток в удаленный репозиторий

```
~/mi/trpp/domain-driven-hexagon  branch2  git push --all
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 16 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 1.10 KiB | 1.10 MiB/s, done.
Total 13 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), completed with 2 local objects.
To https://github.com/mrmin50000/domain-driven-hexagon.git
 * [new branch]      branch1 -> branch1
 * [new branch]      branch2 -> branch2
~/mi/trpp/domain-driven-hexagon  branch2
```

3.7. Выполнение 3-х коммитов в branch1

На рисунке 3.9 приведен вывод истории коммитов после добавления новых 3.

Рисунок 3.9 – история коммитов после добавления 3-х новых

```
commit e893c6d88e7ce9ed433e969abc87b3b9374b81d4 (HEAD -> branch1)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:27:03 2026 +0300

    3

commit dc04458e9fd1c1573a853759ee3c03b29877ddc6
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:26:58 2026 +0300

    w

commit e40d4d51bf4d450f26f01d7b67fb008f430c71d1
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:26:51 2026 +0300

    1

commit a944a6ee64884a79271746bb99a95d8c9ac90ce5 (origin/branch1)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:25 2026 +0300

    3

commit 07893ccdf1c2e12f9c08bb7dcad8de79004fc79f
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:20 2026 +0300

    2

commit b8177f4415b4b047af1177e4549090d3ed62ed2c
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:07 2026 +0300
```

3.8. Повторное клонирование репозитория в другую директорию

Воспользуемся командой *git clone* и клонируем репозиторий в новую директорию, результат выполнения представлен на рисунке 3.10.

Рисунок 3.10 – клонирование репозитория

```
~/mirea/trpp  mkdir test
~/mirea/trpp  cd test
~/mirea/trpp/test  git clone https://github.com/mrmin50000/domain-driven-hexagon.git
Cloning into 'domain-driven-hexagon'...
remote: Enumerating objects: 3373, done.
remote: Counting objects: 100% (607/607), done.
remote: Compressing objects: 100% (112/112), done.
remote: Total 3373 (delta 531), reused 498 (delta 495), pack-reused 2766 (from 2)
Receiving objects: 100% (3373/3373), 7.61 MiB | 401.00 KiB/s, done.
Resolving deltas: 100% (1962/1962), done.
~/mirea/trpp/test
```

3.9. Выполнение 3-х коммитов в branch1

На рисунке 3.11 приведен вывод истории коммитов после добавления новых 3.

Рисунок 3.11 – история коммитов после добавления 3-х новых

```
commit a038282899baaa84b922d167be96591d6f983af8 (HEAD -> branch1)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:29:12 2026 +0300

    3

commit bdb7f7293440c272cafa7fe8ad19488fcdededf4
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:29:00 2026 +0300

    2

commit 49e17c91263095ec94b97c247062b7d6b344f263
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:28:49 2026 +0300

    1

commit a944a6ee64884a79271746bb99a95d8c9ac90ce5 (origin/branch1)
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:25 2026 +0300

    3

commit 07893ccdf1c2e12f9c08bb7dcad8de79004fc79f
Author: mrmin50000 <danilast2007@gmail.com>
Date:   Thu Feb 26 11:23:20 2026 +0300

    2

commit b8177f4415b4b047af1177e4549090d3ed62ed2c
Author: mrmin50000 <danilast2007@gmail.com>
```

3.10. Выгрузка всех изменений в удаленный репозиторий

Выгрузим изменения из 2-й копии репозитория на GitHub с помощью команды *git push*, результат выполнения представлен на рисунке 3.12.

Рисунок 3.12 – выгрузка изменений из 2-й копии репозитория на GitHub

```
~/mi/t/t/domain-driven-hexagon  branch1 ↑3  git push
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 794 bytes | 794.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 2 local objects.
To https://github.com/mrmin50000/domain-driven-hexagon.git
   a944a6e..a038282  branch1 -> branch1
~/mi/t/t/domain-driven-hexagon  branch1
```

3.11. Выгрузка изменений из старого репозитория

Для выгрузки изменений из старого репозитория нам понадобится опция --force. На рисунке 3.13 представлен пример выгрузки без опции --force и с ней.

Рисунок 3.13 – выгрузка изменений из старой копии репозитория на GitHub

```
~/mirea/trpp/domain-driven-hexagon  branch1  git push origin branch1
To https://github.com/mrmin50000/domain-driven-hexagon.git
 ! [rejected]        branch1 -> branch1 (fetch first)
error: failed to push some refs to 'https://github.com/mrmin50000/domain-driven-hexagon.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
~/mi/t/domain-driven-hexagon  branch1  git push --force origin branch1
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 795 bytes | 795.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 2 local objects.
To https://github.com/mrmin50000/domain-driven-hexagon.git
 + a038282...e893c6d branch1 -> branch1 (forced update)
~/mi/trpp/domain-driven-hexagon  branch1
```

3.12. Получение всех изменений в новом репозитории

Для того, чтобы получить все изменения, которые находятся в удаленном репозитории, но отсутствуют в локальном перейдем в этот репозиторий и воспользуемся командой *git pull*, результат выполнения представлен на рисунке 3.14.

Рисунок 3.14 – получение изменений из удаленного репозитория

```
~/mi/trpp/test/domain-driven-hexagon  branch1  git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 443 bytes | 443.00 KiB/s, done.
From https://github.com/mrmin50000/domain-driven-hexagon
 + a038282...e893c6d branch1    -> origin/branch1  (forced update)
```

4. Ответы на контрольные вопросы

1. Какие существуют типы систем контроля версий? Приведите примеры по каждому типу.

Локальные – хранят версии только на локальном компьютере. Пример: RCS (Revision Control System).

Централизованные – все версии хранятся на центральном сервере. Пример: CVS, Subversion.

Распределенные – у каждого клиента полная копия репозитория. Пример: Git

3. Что такое репозиторий Git?

Репозиторий Git – хранилище проекта, которое содержит все файлы и папки проекта, историю изменений, ветки и теги

6. Что такое тег в репозитории Git?

Тег – именованная ссылка на конкретный коммит.

15. Что делает команда git show

Git show показывает информацию об объекте git (коммите, теге, дереве), показывает метаданные, разницу (что изменилось в коммите).

16. Что делает команда git stash?

Git stash временно прячет незакоммиченные изменения, добавленные в staging area и возвращает рабочую директорию к состоянию на момент последнего коммита. Чтобы работать с неотслеживаемыми файлами нужно ввести флаг -u

25. Как отменить действие команды git add на файл?

Для отмены действия команды git add на файл необходимо ввести команду git reset HEAD <имя файла>. Она уберет файлы из staging area, но изменения останутся в нем. Если нужно убрать изменения, то нужно ввести git checkout <имя файла>

29. Как удалить локальную ветку my_branch?

Для удаления локальной ветки нужно воспользоваться командой git branch -d my_branch, если ветка слита и командой git branch -D my_branch,

чтобы принудительно удалить ее.

31. Как удалить все untracked файлы?

Для удаления всех untracked файлов нужно ввести git clean -n, чтобы проверить какие файлы будут удалены, а затем ввести git clean -f, чтобы удалить только файлы. Чтобы удалить папки и файлы нужен флаг -fd, чтобы удалить еще и игнорируемые файлы нужно ввести git clean -fx, чтобы удалить вообще все можно ввести флаг -fdx.

Вывод

По итогам выполнения работы были получены навыки работы с git’ом (создание коммитов, индексация изменений, создание веток и работа с ними), командной строкой, с сервисом для размещения удаленных репозиториев GitHub.